Software Requirements and Design Document

For

Group 2

Version 2.0

Authors:
Colin Maloney
Alex Nino
Kobus VanSteenburg
Samuel Pauly
Zachary Kupersmith

1. Overview (5 points)

Track Tracker is a music streaming data analytics tool. The tool ingests trending sounds from TikTok and corroborates data from Spotify to identify tracks with the potential to become future hits. We are using the TikTok-Api (unofficial) and Spotify Web API for data collection, with web-scraping as needed to supplement. For backend services and data ingestion we are using Python with FastAPI, and for front-end we are using Next.JS with TypeScript. All data is stored in PostgreSQL on AWS RDS.

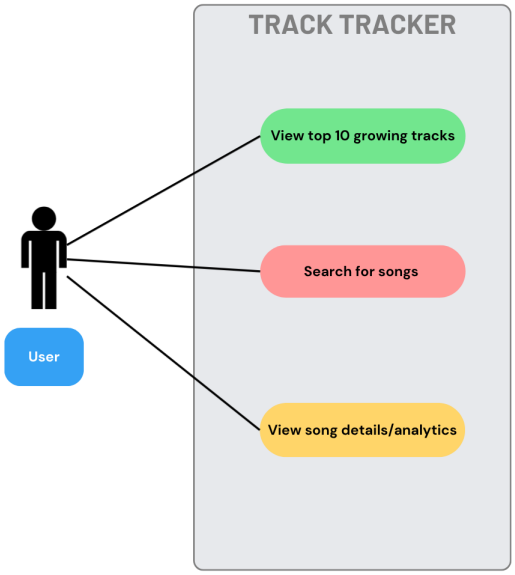
2. Functional Requirements (10 points)

1. High – The system shall search TikTok for videos across a configurable list of music-discovery hashtags (e.g., #newmusic, #indiemusic, #underratedartist).
2. High – The system shall extract sound title and author username from each TikTok video, filtering out original or untitled sounds.
3. High – The system shall deduplicate discovered TikTok sounds by their TikTok sound ID before passing them to Spotify enrichment.
4. High – The system shall search the Spotify Web API using the extracted artist and track names.
5. High – The system shall persist all track, artist, and enrichment data in a PostgreSQL database.
6. High – The system shall prevent duplicate tracks and artists by enforcing uniqueness using the Spotify ID.
7. High – The system shall record a TrackSnapshot on every ingestion run capturing Spotify popularity and a timestamp for time-series momentum tracking.
8. Medium – The system shall expose a REST API via FastAPI with OAuth2 token-based authentication.
9. Medium – The system shall log all ingestion and enrichment failures.

3. Non-functional Requirements (10 points)

1. High – The system shall handle failed API calls gracefully by catching exceptions and continuing pipeline execution without crashing.
2. High – The system shall use database connection pool pre-ping to recover from stale connections after PostgreSQL restarts.
3. Medium – The system shall provide structured logging for all pipeline stages.
4. Medium – The system shall use Alembic for versioned database schema migrations.
5. Medium – The system shall use environment variables for all secrets and connection strings with no hardcoded credentials in the codebase.

4. Use Case Diagram (10 points)



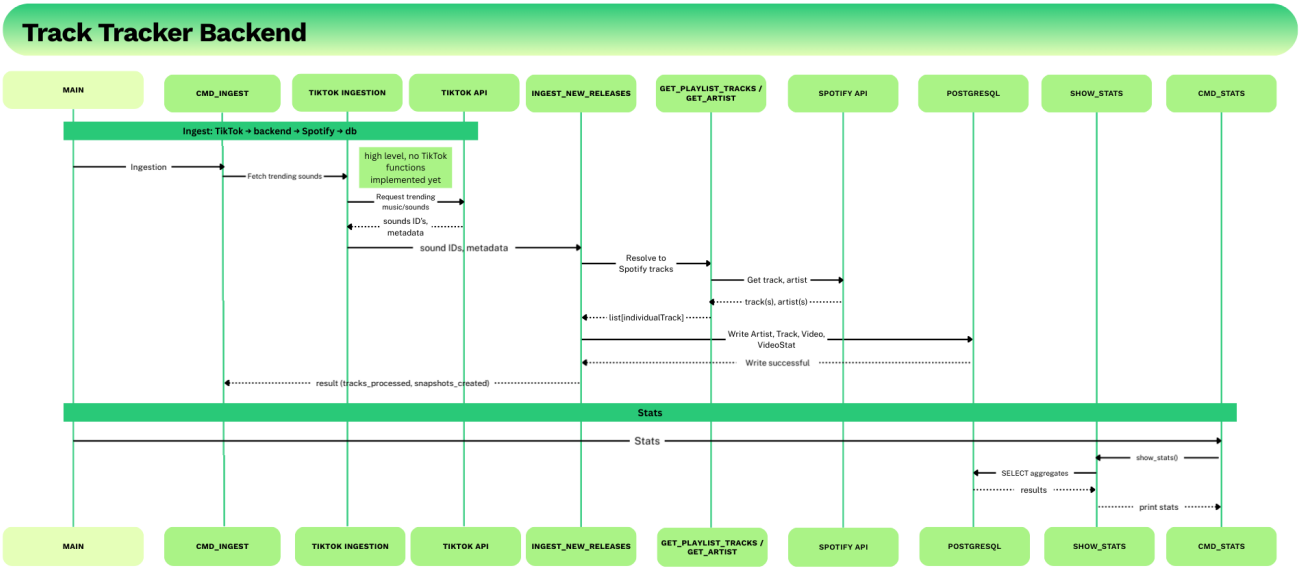
Textual Descriptions:

View top 10 growing tracks: Basically when a user visits the website, the landing page will display the top 10 growing tracks. No preconditions or postconditions.

Search for songs: Users will be able to type songs into the search bar and search our database for them. Expands use beyond top 10 songs. No preconditions or postconditions.

View song details/analytics: Users can view song details on each song card, or they can click the song card and an expandable modal will open that displays more song analytics (e.g. line graph of streams per day)

5. Class Diagram and/or Sequence Diagrams (15 points)



6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

7. Assumptions and Dependencies (5 points)

Dependencies: Unofficial TikTok API

- third party API we are using for TikTok data ingestion.
- Dependent on the API being up to date and running correctly